

university of groningen

Performance Engineering: Lecture #07

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-09-23

Overview



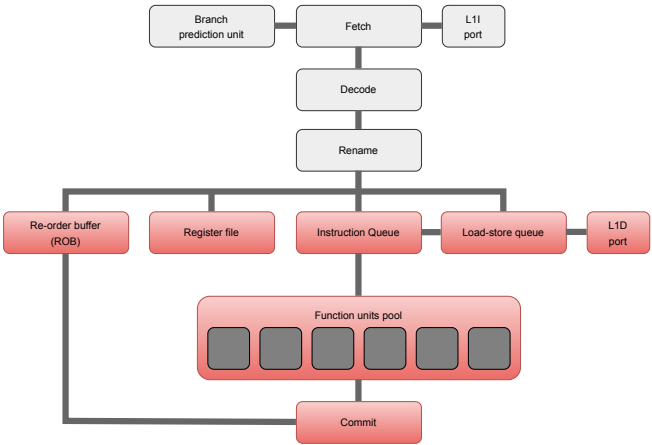
1. Processor Microarchitecture: Back-end
2. Processor Microarchitectures
3. Machine Code Analysis: `llvm-mca`
4. Top-down Microarchitecture Analysis
5. Loop Optimizations (1)
6. Final

Content



1. **Processor Microarchitecture: Back-end**
2. Processor Microarchitectures
3. Machine Code Analysis: `llvm-mca`
4. Top-down Microarchitecture Analysis
5. Loop Optimizations (1)
6. Final

Microarchitecture Overview: Back-End



In-Order and Out-of-Order



- Back-end instruction processing may be **in-order** or **out-of-order**
- **Out-of-order instruction processing** = Allow instructions to be executed in an order differently from program order
  - The microarchitecture must guarantee that execution appears like instructions being executed in the original order
  - Reordering depth depends on microarchitecture implementation
  - The front-end is always in-order
- **Performance engineering potential**: Depends on re-ordering depth

Execution Units



- Challenge of **unfilled execution pipelines**
  - Typical issue: data hazards that cause instructions to be stalled
- Modern microarchitectures feature a large number of execution units
  - Exploiting **instruction-level parallelism** is key
  - Relevant metric: **instructions per cycle (IPC)**
- Different execution units have different characteristics
  - Typically different depth
  - Some execution units may not support pipelining
- **Performance engineering potential**: High

Data hazards



- Assume that in program order instruction *i* occurs before instruction *j*
- The instruction *j* is **data dependent** on the instruction *i* if either
  - Instruction *i* produces a result that may be used by instruction *j*
  - Instruction *j* is data dependent on instruction *k*, and instruction *k* is data dependent on instruction *i*
- **Read-after-write (RAW) hazard**
  Instruction *j* tries to read a source before it is updated by instruction *i*

  Example for wrong scheduling of instructions:

```
imul R0, R1, R5 !
isub R6, R3, R7 ! Register R6 not written, yet
iadd R5, R2, R6 !
```

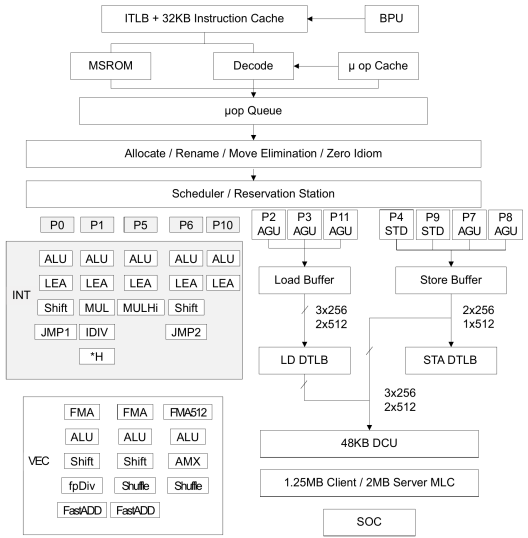
Content



1. Processor Microarchitecture: Back-end
2. Processor Microarchitectures
3. Machine Code Analysis: `llvm-mca`
4. Top-down Microarchitecture Analysis
5. Loop Optimizations (1)
6. Final

Intel Golden Cove (1/2)

[Intel, “Optimization Reference Manual”, v49]



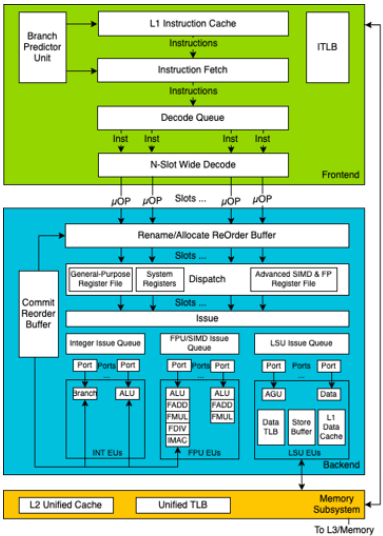
Intel Golden Cove (2/2)



- Front-end fetch bandwidth: $32\,\mathrm{B/cycle}$
- Decoder: 6 instructions/cycle
- Reservation station capacity: 144 micro-instructions
- Number of execution ports: 12
- Floating-point arithmetics latency: [Intel; March 2022]
  - Addition (`ADDPD`): 3 cycles
  - Multiplication (`MULSD`): 4 cycles

Arm Neoverse v2 (1/2)

["Arm Neoverse V2 Core Telemetry Specification", version 2.0]



Arm Neoverse v2 (2/2)



- Front-end fetch and decode bandwidth: implementation specific
- Decoder: 6 instructions/cycle
- Reservation station capacity: implementation specific
- Number of execution ports: 17
- Floating-point arithmetics latency: ["Arm Neoverse V2 Software Optimization Guide", December 2022]
  - Addition (`FADD`): ≥ 2 cycles
  - Multiplication (`FMUL`): ≥ 3 cycles
  - Multiply-accumulate (`FMADD`): ≥ 4 cycles

Content



1. Processor Microarchitecture: Back-end
2. Processor Microarchitectures
3. **Machine Code Analysis: `llvm-mca`**
4. Top-down Microarchitecture Analysis
5. Loop Optimizations (1)
6. Final

`llvm-mca`: LLVM Machine Code Analyzer



- Tool to **statically model** the performance of machine code
  - Given an assembly code sequence, the tool estimates the IPC, hardware resource pressure, and other quantities
  - Requires model to be available in LLVM
  - Code may be compiled with GCC
- **Analysis steps**:
  - Instruction information (default)
  - Resource pressure (default)
  - Throughput Bottlenecks (`-bottleneck-analysis`)
  - Critical sequence (`-bottleneck-analysis`)
- **Choice of architecture**:
  - `-mtriple=<target triple>`: `x86_64-unknown-unknown`, `aarch64-unknown-unknown`
  - `-mcpu=<cpu>`: Specify the processor for which to analyse the code (use `help` to list supported processors)

Example Code


```
1  double a[8], b[8], c[8];
2
3  void kernel(void) {
4      c[0] = a[0] + b[0];
5      c[1] = a[1] + b[1];
6      c[2] = a[2] + b[2];
7      c[3] = a[3] + b[3];
8      c[4] = a[4] + b[4];
9      c[5] = a[5] + b[5];
10     c[6] = a[6] + b[6];
11     c[7] = a[7] + b[7];
12 }
```

Compiler command line: `gcc-14 -O1`

`llvm-mca` on x86 (1/4)



Tool command line: `llvm-mca-20 \`
`-mtriple=x86_64-unknown-unknown \`
`-mcpu=tigerlake \`
`-bottleneck-analysis`

Header:

```
1 Iterations:        100
2 Instructions:      2600
3 Total Cycles:      1353
4 Total uOps:        4400
5
6 Dispatch Width:    6
7 uOps Per Cycle:    3.25
8 IPC:               1.92
9 Block RThroughput: 8.5
```

`llvm-mca` on x86 (2/4)



Instruction information:

```
Instruction Info:
[1]: #uOps
[2]: Latency
[3]: RThroughput
[4]: MayLoad
[5]: MayStore
[6]: HasSideEffects (U)

[1]    [2]    [3]    [4]    [5]    [6]    Instructions:
 1      100    0.25                  U     endbr64
 1      5      0.50    *                   movsd a(%rip), %xmm0
 2      9      0.50    *                   addsd b(%rip), %xmm0
 2      1      0.50            *           movsd %xmm0, c(%rip)
 1      5      0.50    *                   movsd 8+a(%rip), %xmm0
 2      9      0.50    *                   addsd 8+b(%rip), %xmm0
 2      1      0.50            *           movsd %xmm0, 8+c(%rip)
```

`llvm-mca` on x86 (3/4)



Resource pressure:

```
89  Resources:
90  [0]   - ICXDivider
91  [1]   - ICXFPDivider
92  [2]   - ICXPort0
93  [3]   - ICXPort1
94  [4]   - ICXPort2
95  [5]   - ICXPort3
96  [6]   - ICXPort4
97  [7]   - ICXPort5
98  [8]   - ICXPort6
99  [9]   - ICXPort7
100 [10]  - ICXPort8
101 [11]  - ICXPort9
102
103
104 Resource pressure per iteration:
105 [0]    [1]    [2]    [3]    [4]    [5]    [6]    [7]    [8]    [9]    [10]   [11]
106 -      -      4.43   4.44   8.50   8.50   4.00   1.00   1.13   4.00   4.00   4.00
107
108 Resource pressure by instruction:
109 [0]    [1]    [2]    [3]    [4]    [5]    [6]    [7]    [8]    [9]    [10]   [11]   Instructions:
110 -      -      -      0.87   -      -      -      -      0.13   -      -      -      endbr64
111 -      -      -      -      0.50   0.50   -      -      -      -      -      -      movsd      a(%rip), %xmm0
112 -      -      0.93   0.07   0.50   0.50   -      -      -      -      -      -      addsd      b(%rip), %xmm0
113 -      -      -      -      -      -      -      -      -      -      1.00   1.00   movsd      %xmm0, c(%rip)
```

`llvm-mca` on x86 (4/4)



Bottleneck analysis:

```
12 Cycles with backend pressure increase [ 13.16% ]
13 Throughput Bottlenecks:
14   Resource Pressure       [ 5.76% ]
15   - ICXPort2  [ 5.76% ]
16   - ICXPort3  [ 5.76% ]
17   Data Dependencies:      [ 13.16% ]
18   - Register Dependencies [ 13.16% ]
19   - Memory Dependencies   [ 0.00% ]
```

`llvm-mca` on Arm (1/3)



Tool command line: `llvm-mca-20 \`
`-mtriple=aarch64-unknown-unknown \`
`-mcpu=neoverse-v2 \`
`-bottleneck-analysis`

Header:

```
1 Iterations:        100
2 Instructions:      2500
3 Total Cycles:      412
4 Total uOps:        3700
5
6 Dispatch Width:    16
7 uOps Per Cycle:    8.98
8 IPC:               6.07
9 Block RThroughput: 2.7
```

`llvm-mca` on Arm (2/3)

Instruction information:

```
63 Instruction Info:
64 [1]: #uOps
65 [2]: Latency
66 [3]: RThroughput
67 [4]: MayLoad
68 [5]: MayStore
69 [6]: HasSideEffects (U)
70
71 [1]    [2]    [3]    [4]    [5]    [6]    Instructions:
72  1      1     0.25                        adrp   x0, .LANCHOR0
73  1      1     0.17                        add    x0, x0, :lo12:.LANCHOR0
74  1      1     0.17                        add    x2, x0, #64
75  1      1     0.17                        add    x1, x0, #128
76  2      6     0.33    *                   ldp    d31, d30, [x0, #64]
77  2      6     0.33    *                   ldp    d28, d29, [x0, #128]
78  1      2     0.25                        fadd   d31, d31, d28
79  1      2     0.25                        fadd   d30, d30, d29
80  2      2     0.50           *            stp    d31, d30, [x0]
```

`llvm-mca` on Arm (3/3)



Resource pressure:

```
Resources:
[0.0] - V2UnitB
[0.1] - V2UnitB
[1.0] - V2UnitD
[1.1] - V2UnitD
[2.0] - V2UnitFlg
[2.1] - V2UnitFlg
[2.2] - V2UnitFlg
[3]   - V2UnitL2
[4.0] - V2UnitL01
[4.1] - V2UnitL01
[5]   - V2UnitM0
[6]   - V2UnitM1
[7]   - V2UnitS0
[8]   - V2UnitS1
[9]   - V2UnitS2
[10]  - V2UnitS3
[11]  - V2UnitV0
[12]  - V2UnitV1
[13]  - V2UnitV2
[14]  - V2UnitV3


Resource pressure per iteration:
[0.0]  [0.1]  [1.0]  [1.1]  [2.0]  [2.1]  [2.2]  [3]    [4.0]  [4.1]  [5]    [6]    [7]    [8]    [9]    [10]   [11]   [12]   [13]   [14]
0.50   0.50   -      -      -      -      -      3.97   4.01   4.02   0.66   0.66   0.67   0.67   0.67   0.67   3.00   3.01   2.99   3.00

Resource pressure by instruction:
[0.0]  [0.1]  [1.0]  [1.1]  [2.0]  [2.1]  [2.2]  [3]    [4.0]  [4.1]  [5]    [6]    [7]    [8]    [9]    [10]   [11]   [12]   [13]   [14]   Instructions:
-      -      -      -      -      -      -      -      -      -      -      0.33   -      0.67   -      -      -      -      -      -      adrp    x0, .LANCHOR0
-      -      -      -      -      -      -      -      -      -      0.33   -      0.33   -      -      0.34   -      -      -      -      add x0, x0, :lo12:.LANCHOR0
-      -      -      -      -      -      -      -      -      -      -      0.33   -      -      0.34   0.33   -      -      -      -      add x2, x0, #64
```

Content



1. Processor Microarchitecture: Back-end
2. Processor Microarchitectures
3. Machine Code Analysis: `llvm-mca`
4. **Top-down Microarchitecture Analysis**
5. Loop Optimizations (1)
6. Final

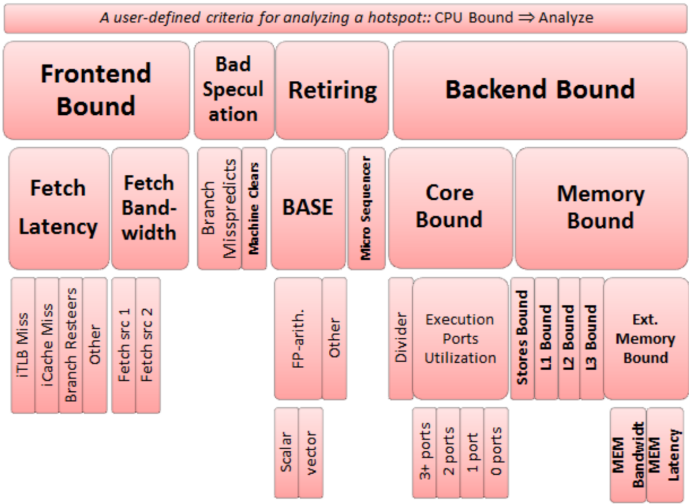
Top-Down Microarchitecture Analysis (TMA)



- **Idea**: Identify and group hardware performance counters to facilitate a top-down microarchitecture performance analysis
- This strategy has first been implemented for Intel processors with the following **top-level analysis categories**: [A. Yasin; 2014]
  - **Front-end bound**: Fraction of slots where front-end undersupplies back-end
  - **Bad speculation**: Fraction of slots wasted due to incorrect speculations
  - **Retiring**: Fraction of slots utilized by useful work (committed instructions)
  - **Back-end bound**: Fraction of slots where no micro-instructions are being delivered due to a lack of required resources

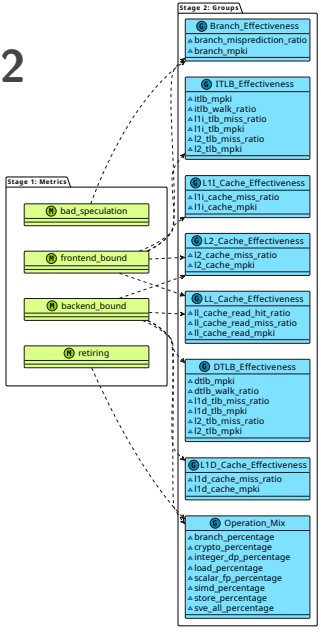
TMA on Intel Processors

[A. Yasin; 2014]



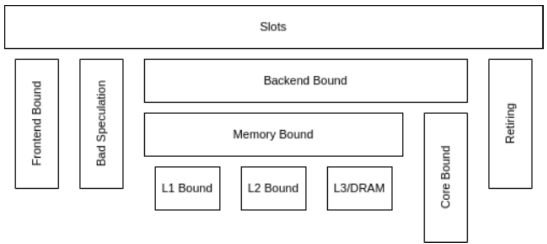
TMA on Arm Neoverse v2

"Arm Neoverse V2 Core Telemetry Specification", version 2.0



TMA on Kunpeng 920

[F. Banchelli et al.; 2023]



Front-end bound	$\texttt{FETCH_BUBBLE}/(4 \cdot \texttt{CPU_CYCLES})$
Bad speculation	$(\texttt{INST_SPEC} - \texttt{INST_RETIRED})/(4 \cdot \texttt{CPU_CYCLES})$
Retiring	$\texttt{INST_RETIRED}/(4 \cdot \texttt{CPU_CYCLES})$
Backend bound	1 - (Front-end bound + Bad speculation + Retiring)

TMA using `perf` (1/2)



Example code:

```c
double a[8], b[8], c[8];

void kernel(void) {
    c[0] = a[0] + b[0];
    c[1] = a[1] + b[1];
    c[2] = a[2] + b[2];
    c[3] = a[3] + b[3];
    c[4] = a[4] + b[4];
    c[5] = a[5] + b[5];
    c[6] = a[6] + b[6];
    c[7] = a[7] + b[7];
}

int main() {
    for (int i = 0; i < 100000000; i++)
        kernel();
}
```

Compiler command: `gcc-14 -O1`

TMA using `perf` (2/2)



Perf output on Intel i7-1165G7 (Tiger Lake):

```
Performance counter stats for './kernel1-x86.x':

            252.10 msec task-clock                #    0.998 CPUs utilized
                 2      context-switches          #    7.933 /sec
                 2      cpu-migrations            #    7.933 /sec
                54      page-faults               #  214.201 /sec
     1,154,029,248      cycles                    #    4.578 GHz
     2,902,969,578      instructions              #    2.52  insn per cycle
       300,499,859      branches                  #    1.192 G/sec
            10,538      branch-misses             #    0.00% of all branches
                        TopdownL1                 #     47.0 %  tma_backend_bound
                                                  #      1.2 %  tma_bad_speculation
                                                  #      0.1 %  tma_frontend_bound
                                                  #     51.7 %  tma_retiring

       0.252639477 seconds time elapsed

       0.252563000 seconds user
       0.000000000 seconds sys
```

Content



1. Processor Microarchitecture: Back-end
2. Processor Microarchitectures
3. Machine Code Analysis: `llvm-mca`
4. Top-down Microarchitecture Analysis
5. **Loop Optimizations (1)**
6. Final

Introduction to Loop Optimization



- Loop optimization is a typical performance engineering effort
- The following loop optimization techniques will be covered:
  - Loop fusion
  - Loop unroll
  - Loop unroll and jam
- The following analysis techniques will be introduced:
  - Identification of loop carry dependencies
  - Critical path analysis

Case Study: `field_copy`



- **Purpose** of this case study: Explore impact of loop fusion
- **Specification** for `field_copy`:
  - Copy a field of 3-dimensional position variables
  - The variables are double-precision numbers

`field_copy`: Initial Implementation



```c
#include <stdlib.h>

typedef struct {
  double x;
  double y;
  double z;
} pos_t;

void field_copy(pos_t *restrict a,
                pos_t *restrict b,
                int n) {
  for (int i = 0; i < n; i++)
    a[i].x = b[i].x;

  for (int i = 0; i < n; i++)
    a[i].y = b[i].y;

  for (int i = 0; i < n; i++)
    a[i].z = b[i].z;
}
```

```c
int main() {
  const int n = 1048576;
  pos_t *a   = (pos_t *) calloc(n, sizeof(pos_t));
  pos_t *b   = (pos_t *) calloc(n, sizeof(pos_t));

  for (int i = 0; i < 1000; i++)
    field_copy(a, b, n);
}
```

Loop Fusion



- **Loop fusion** = Loop optimization technique where separate loops are merged
- **Prerequisites**:
  - All loops iterate over the same range
  - Loops do not reference each other's data
- **Target benefit(s)**
  - Reduce loop control overhead (branching, condition checks)
  - Enable further optimizations within larger basic blocks
  - Reduced data transfer
- **Possible drawback(s)**
  - Large code size
  - More instruction cache misses

`field_copy`: Fused Loops



```c
#include <stdlib.h>

typedef struct {
  double x;
  double y;
  double z;
} pos_t;

void field_copy_fused(
        pos_t *restrict a,
        pos_t *restrict b,
        int n) {
  for (int i = 0; i < n; i++) {
    a[i].x = b[i].x;
    a[i].y = b[i].y;
    a[i].z = b[i].z;
  }
}
```

```c
int main() {
  const int n = 1048576;
  pos_t *a   = (pos_t *) calloc(n, sizeof(pos_t));
  pos_t *b   = (pos_t *) calloc(n, sizeof(pos_t));

  for (int i = 0; i < 1000; i++)
    field_copy_fused(a, b, n);
}
```

`field_copy`: Kunpeng 920 Results

	N_{cycle}	N_{instr}
Original, `-O1`	$14.52 \cdot 10^9$	$15.73 \cdot 10^9$
Fused, `-O1`	$3.44 \cdot 10^9$	$8.39 \cdot 10^9$
Original, `-O3`	$2.62 \cdot 10^5$	$1.55 \cdot 10^5$
Fused, `-O3`	$2.57 \cdot 10^5$	$1.55 \cdot 10^5$

- **Compiler**: GCC 14.1.0
- **Observations**:
  - Using compiler optimization level `-O1`: Number of cycles and instructions improve by $4.2\times$ and $1.9\times$
  - Using compiler optimization level `-O3`: Explicit optimization is not required

Content



1. Processor Microarchitecture: Back-end
2. Processor Microarchitectures
3. Machine Code Analysis: `llvm-mca`
4. Top-down Microarchitecture Analysis
5. Loop Optimizations (1)
6. **Final**

Summary Lecture #07



- Review of key elements of processor microarchitectures with the following having the highest performance engineering potential
  - Back-end: Out-of-order execution, execution units
- Overview of different processor microarchitectures:
  - Intel Golden Cove
  - Arm Neoverse v2
- Static code analysis using `llvm-mca`
- Dynamic top-down microarchitecture analysis (TMA):
  - Front-end bound
  - Bad speculation
  - Retiring
  - Back-end bound
- Loop optimizations: Loop fusion

Questions?



[Wiki Media]